

CRC Transmitter Functional Design Specifications

CSI-2 Controller IP

Version 4.0.1

Block Owner

Mina Nasser

Authors

Mina Nasser

Contents

1	Revision History	3
2	Overview	4
3	Operation and Description	4
3.1	Digital Interface	4
3.1.1	Parameters Names	4
3.1.2	Ports Names	4
3.2	Functional Description	4
3.3	Block Diagram	5
3.4	Timing Diagram	6
3.5	Verification Requirements	6

1 Revision History

The following table:

Version	Date	Author(s)	Revision Notes	Owner Approval
1	2025-12-14	Mina Nasser	Initial FDS creation for crc_tx module.	

2 Overview

The `crc_tx` (CRC Transmitter) block is a digital logic module designed to ensure data integrity within the CSI-2 transmission protocol.

It performs a cyclic redundancy check (CRC) calculation on an incoming stream of 8-bit payload data. The module implements a 16-bit CRC algorithm. Upon the completion of a data packet (signaled by an End-of-Transmission flag), the module appends the calculated 16-bit Checksum to the output stream.

The checksum is transmitted as two sequential bytes: the Least Significant Byte (LSB) followed by the Most Significant Byte (MSB).

3 Operation and Description

3.1 Digital Interface

3.1.1 Parameters Names

There are no configurable parameters exposed in the module interface.

3.1.2 Ports Names

The following table:

Port Name	Width	Type	Description
<code>clk</code>	1	Input	System Clock. Trigger on rising edge.
<code>reset_n</code>	1	Input	Active-low async reset. Resets CRC to 0xFFFF.
<code>i_data_valid</code>	1	Input	Indicates <code>i_data_in</code> is valid payload.
<code>i_data_in</code>	8	Input	Payload byte input stream.
<code>i_start</code>	1	Input	Start of Transmission (SoT). Resets CRC.
<code>i_end</code>	1	Input	End of Transmission (EoT). Last byte flag.
<code>o_valid</code>	1	Output	Valid high during CRC transmission.
<code>o_data</code>	8	Output	CRC Output bytes (LSB then MSB).

3.2 Functional Description

The `crc_tx` block implements a standard CRC-16 Checksum managed by an internal Finite State Machine (FSM):

1. Initialization Phase

When `i_start` is asserted, the internal CRC state is forced to the initialization value 0xFFFF. If `i_data_valid` is also High, the logic initializes and immediately processes the incoming byte in the same clock cycle (Sync Start).

2. Data Processing Phase

For every valid input byte, the module performs a "Reflected Update":

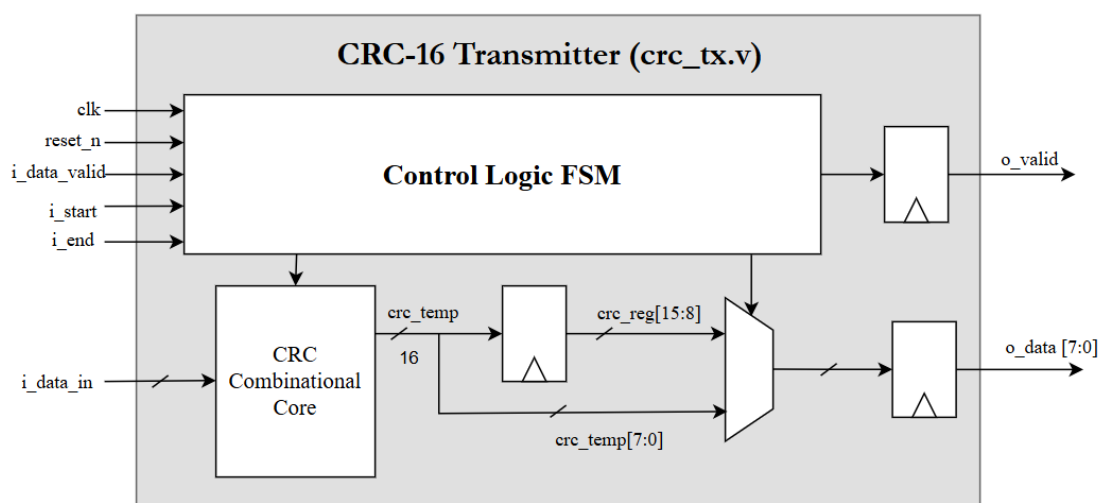
- The current CRC state (16-bit) is XORed with the input data (8-bit).
- The result passes through a combinational chain of 8 XOR/Shift stages (unrolled loop).
- The result is stored back into the `crc_reg` register at the next clock edge.

3. Output Phase (Look-Ahead Mechanism)

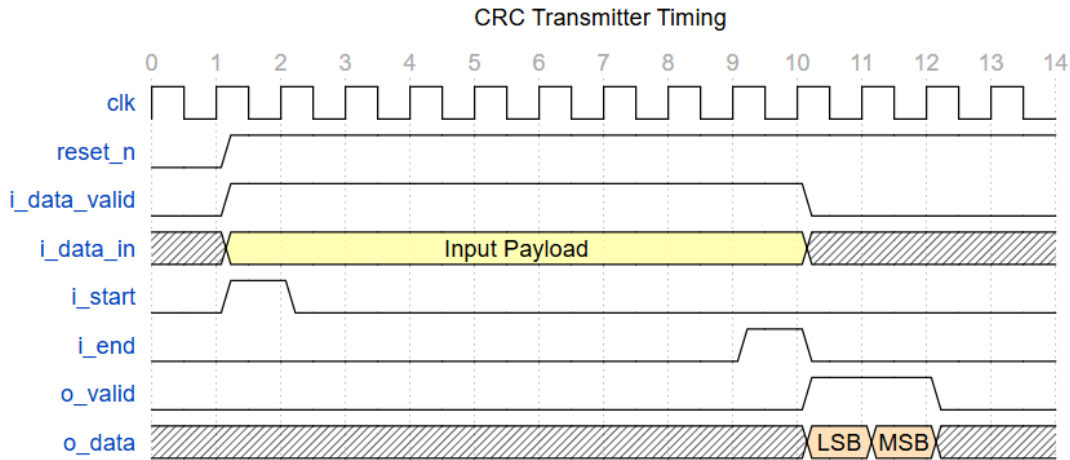
To achieve zero latency:

- **Cycle N (Last Payload Byte):** When `i_end` is detected, the module taps into the combinational output of the CRC calculation (`crc_temp`) before it is registered. It bypasses the flip-flops to drive `o_data` with the LSB immediately.
- **Cycle N+1 (CRC MSB):** The module transitions to an output state (`out_state` = 2). It drives `o_data` with the MSB stored in the register (`crc_reg[15:8]`).
- **Cycle N+2:** The module returns to Idle.

3.3 Block Diagram



3.4 Timing Diagram



3.5 Verification Requirements

The following verification requirements define the functional and timing behavior of the `crc_tx` module as verified using the `tb_crc_tx` testbench.

1. **Reset Functionality (VR-1):**

The design under test (DUT) shall enter a known idle state when the active-low reset signal (`reset_n`) is asserted. During reset, no CRC output shall be generated, and all internal CRC computation logic shall be cleared.

2. **Input Data Validity (VR-2):**

The DUT shall sample input data only when the input valid signal (`i_data_valid`) is asserted. Input data shall be ignored when the valid signal is deasserted.

3. **Start-of-Frame Detection (VR-3):**

The DUT shall detect the start of a new CRC frame when the start signal (`i_start`) is asserted. The CRC calculation shall be reset at the beginning of each new frame.

4. **End-of-Frame Detection (VR-4):**

The DUT shall detect the end of a payload frame when the end signal (`i_end`) is asserted. CRC generation shall occur only after the final data byte of the frame has been processed.

5. **CRC Output Valid Signaling (VR-5):**

The DUT shall assert the output valid signal (`o_valid`) only when a valid CRC value is available. Exactly one CRC output shall be produced per payload frame.

6. **CRC Output Correctness (VR-6):**

The CRC output data (`o_data`) shall match the expected CRC value corresponding to the transmitted payload data sequence.

7. **Multiple Frame Support (VR-7):**

The DUT shall correctly process multiple payload frames sequentially without requiring a reset between frames. CRC computation shall not accumulate across frames.

8. **Variable Payload Length Support (VR-8):**

The DUT shall correctly compute CRC values for payloads of varying lengths, including both single-byte and multi-byte payloads.

9. **Synchronous Operation (VR-9):**

All DUT inputs and outputs shall be synchronous to the rising edge of the clock. No combinational glitches shall be observed on the CRC output signals.

10. **Simulation Completion (VR-10):**

The DUT shall complete all defined test cases without error or deadlock, and the simulation shall terminate successfully after all payloads are processed.